

AI QA Agent — Current Workflow & Optimization Plan

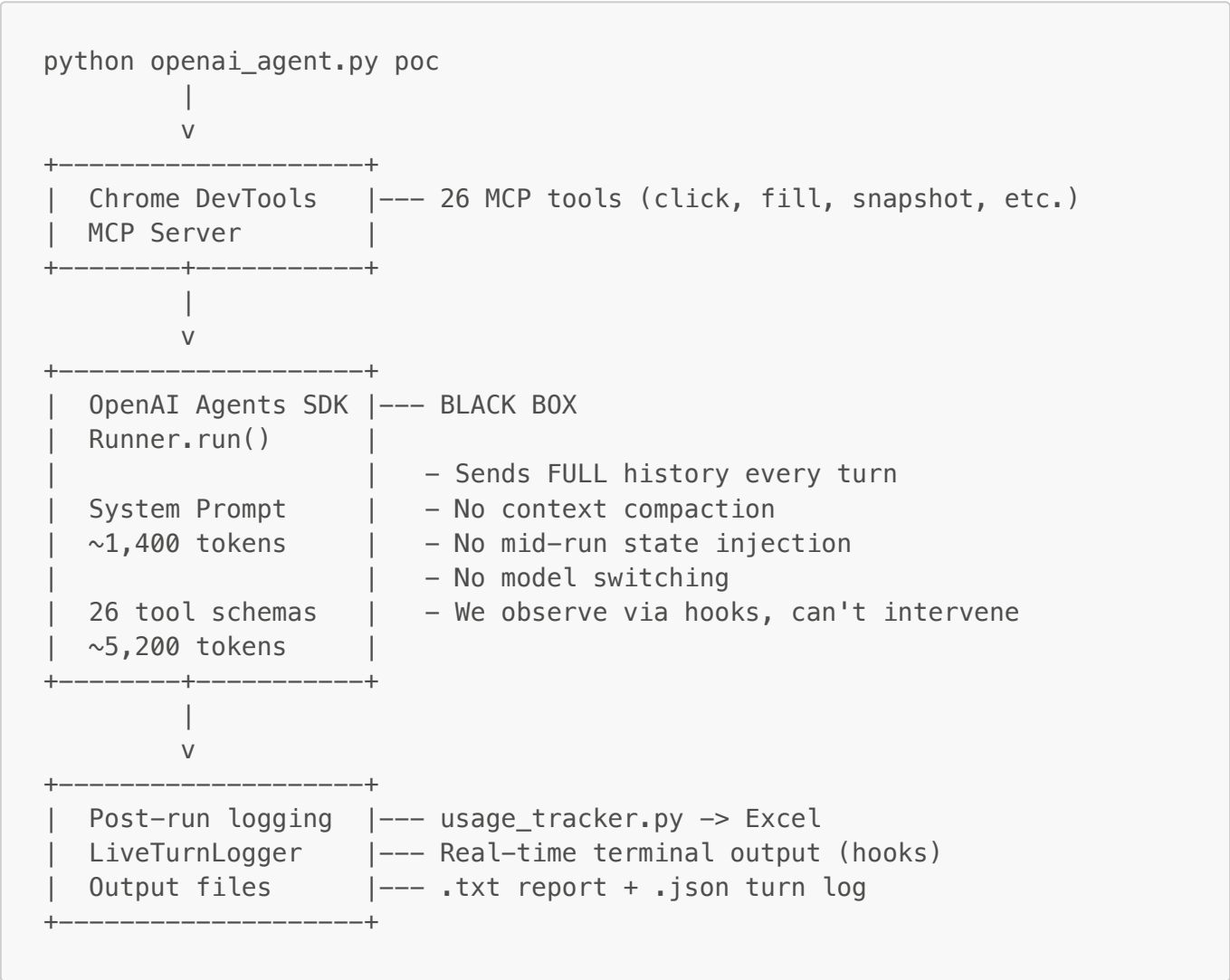
What This Is

An AI agent that autonomously tests web applications. Given a URL, it opens Chrome, explores the page, fills forms, clicks buttons, finds bugs — no scripts, no recordings.

Stack: Python + OpenAI Agents SDK + Chrome DevTools MCP + GPT-5

Current Target App: TECU Credit Union loan form — 6-page wizard, 153 steps

Current Architecture



Files

File	Role
openai_agent.py	Main runner — SDK wiring, hooks, output
prompts/system_prompt.py	System prompt (~1,400 tokens, 5 worked examples)

File	Role
state/tracker.py	StateTracker — built but NOT wired into agent
state/usage_tracker.py	Cost/token logging to Excel
dashboard.py	Streamlit analytics dashboard

What the Agent Does (POC — Page 1 only)

- 1. Opens URL in Chrome
- 2. Takes accessibility snapshot (sees element tree, not pixels)
- 3. Discovers fields: profile photo, first name, middle name, email, mobile, last name, branch
- 4. Fills each field with test data
- 5. Reports results with XPaths, issues found

Cost Model

GPT-5 pricing:

Token Type	Rate (\$/MTok)	Per-turn avg
Uncached input	\$1.25	~2K tokens
Cached input	\$0.125 (90% discount)	~28K tokens
Output	\$10.00	~250 tokens

Per-turn input composition (sent every turn):

system_prompt	~1,400 tok	-- fixed, always cached after turn 2
tool_schemas	~5,200 tok	-- 26 MCP tools, always cached
task_message	~30 tok	-- fixed
history + results	GROWS	-- all previous turns' thoughts + tool calls

Cost Breakdown — Latest Run (46 turns, Page 1)

Component	Tokens	Cost	%
Uncached input	~93K	\$0.116	27%
Cached input	~1.28M	\$0.160	38%
Output	~11.5K	\$0.115	27%
Total		\$0.425	

Cache hit rate: 93.5% | Duration: 270s | Savings vs no-cache: \$1.58 (78.8%)

Example Turn Log (from 46-turn run)

Turn	Action	Target	Input	Cached	New	Out	
Cache%							
-----	-----	-----	-----	-----	-----	-----	-----
1	new_page	https://	6,689	0	---	499	
0%							
2	take_snapshot		7,229	7,168	540	33	
99%							
3	upload_file	1_79	9,375	7,168	2,146	1,521	76%
<--	verbose reflection						
...							
12	fill	1_93	16,415	15,616	855	135	
95%	<-- First Name						
13	fill	1_110	17,261	16,512	846	56	
96%	<-- Middle Name						
14	fill	1_102	17,332	17,280	71	56	
100%	<-- Email						
15	fill	1_126	17,403	17,280	71	84	
99%	<-- Mobile						
16	fill	1_134	17,502	17,408	99	41	
100%	<-- Last Name						
...							
26	click	8_2	48,123	28,288	19,761	290	
59%	<-- country dropdown: +19K tokens!						
...							
46	text_output		53,654	53,376	672	3,398	
100%	<-- final report						

Key observations:

- Turns 12-16 (simple fills): ~70-850 new tokens each, 95-100% cache hit — efficient
- Turn 26 (country dropdown): +19,761 new tokens — ~190 country options inflated context permanently
- Turn 29: cache dropped to 26% — likely cache eviction after large context shift
- Context grew from 6,689 (turn 1) to 53,654 (turn 46)

Where Turns Were Wasted (46 vs target ~28)

Problem	Turns Used	Wasted	What Happened
Photo upload	3-11 (9 turns)	~5	Tried visible button 3x, then image, then found hidden input
Country code	22-27 (6 turns)	~6	Opened dropdown to "test" already-correct default (+1868)

Problem	Turns Used	Wasted	What Happened
Last Name fix	32-41 (10 turns)	~5	type_text caused duplication, multiple retry cycles
Efficient work	remaining (21)	0	Fills, clicks, snapshots — all necessary

Issues Found by Agent (Real Bugs)

- 1. **Last Name input duplication bug** — Typing via keyboard produces garbled text ("TEESTRTTESETESTER"). Had to set value programmatically. Likely a buggy onKey handler.
- 2. **Upload button not wired** — The visible "Add profile picture" button doesn't trigger file chooser. Hidden `<input type="file">` exists separately. Automation-incompatible.
- 3. **Masked mobile input** — `fill()` doesn't update the underlying value for masked inputs. Only keyboard typing works. Common issue with input mask libraries.
- 4. **Accessibility warnings** — Missing autocomplete attributes, form fields without id/name, incorrect label-for usage.

Run History

Date	Turns	Cost	Cache Hit	Duration	Notes
Mar 6 (run 1)	27	\$0.27	~85%	~180s	Page 1, clean run
Mar 6 (run 2)	30	\$0.25	~88%	~190s	Page 1, similar
Mar 7	46	\$0.43	93.5%	270s	Page 1, too many turns

Yesterday: 2 runs for ~\$0.52 total. Today: 1 run for \$0.43 (same work, more turns).

Proposed Optimizations — 5 Levers

Lever 1: Turn Reduction (prompt changes)

Effort: Low | Savings: \$0.10-0.15/page

- Skip elements with correct defaults (don't re-test country code)
- Go straight to hidden `<input type="file">` for uploads
- If `fill()` fails on masked input, jump to `evaluate_script` — skip type_text/press_key cycles

Lever 2: Output Verbosity (prompt changes)

Effort: Low | Savings: \$0.03-0.05/page

Output costs \$10/MTok — most expensive token type. Currently the prompt encourages verbose THOUGHT/REFLECTION/INTERPRETATIONS on every action. Change to: concise reasoning for simple

actions, detailed reasoning only on failures.

Lever 3: Snapshot Bloat Control (prompt changes)

Effort: Low | **Savings:** \$0.02-0.03 per large dropdown

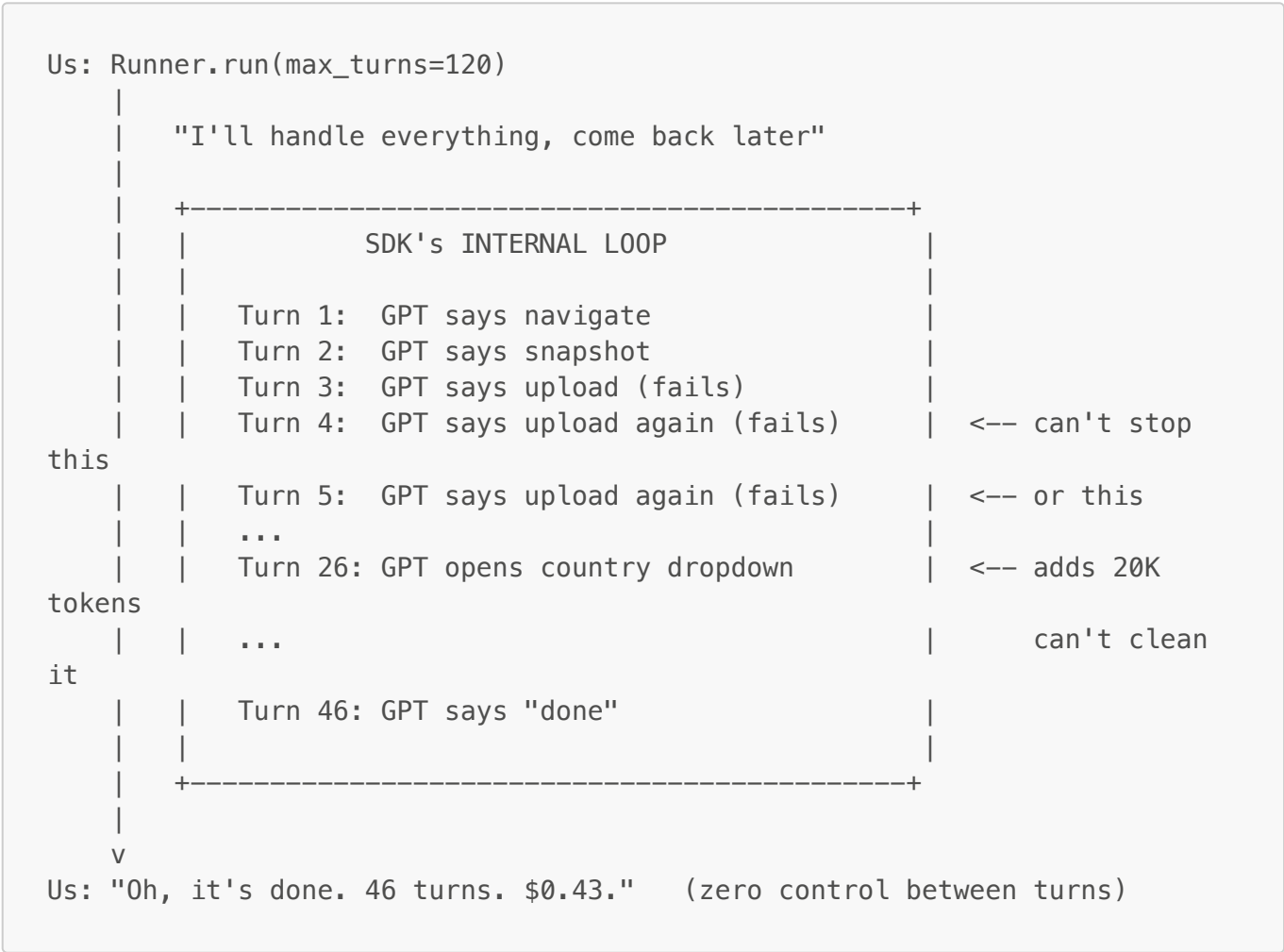
Country dropdown added 19,761 tokens permanently. Fix: close dropdowns before snapshotting, or use `evaluate_script` to query specific options instead of full snapshots.

Lever 4: Orchestrated Turn Loop with Context Compaction

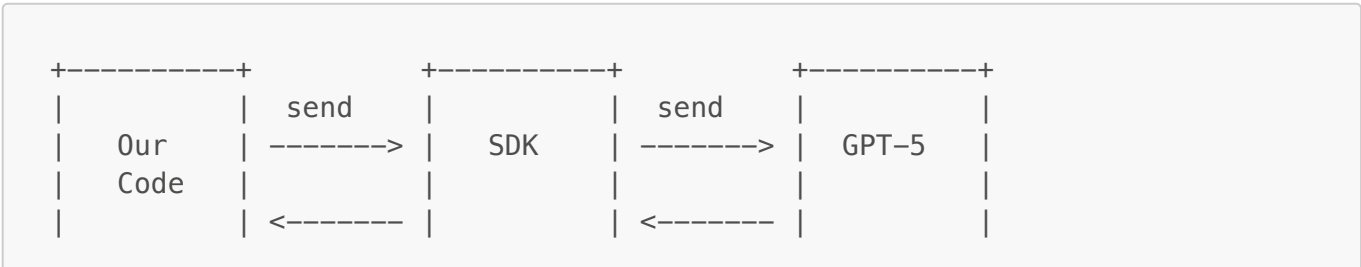
Effort: Medium | **Savings:** 20-40% on multi-page runs

The agent still decides everything (what to click, fill, diagnose). We just manage its memory between turns — cleaning out old data and keeping a rolling summary.

Current flow — SDK controls the loop, memory piles up:



Proposed flow — We control the loop, SDK runs one turn at a time:



```
|         | receive |         | receive |         |
+-----+ +-----+ +-----+

Turn 1:
Us --> SDK: "here's the task"
SDK --> GPT: sends task to GPT-5
GPT --> SDK: "navigate to URL"
SDK --> Us: gives back result
Us: updates summary: "Navigated to URL"
Us: passes summary + latest state to SDK for next turn

Turn 2:
Us --> SDK: "here's summary + latest state"
SDK --> GPT: sends it to GPT-5
GPT --> SDK: "take snapshot"
SDK --> Us: gives back result
Us: updates summary: "Navigated. Snapshot taken. Found 7 fields."
Us: passes summary + latest snapshot to SDK for next turn

Turn 12:
Us --> SDK: "here's summary + latest state"
GPT --> SDK: "fill First Name = Roman"
SDK --> Us: gives back result
Us: updates summary: "Photo done. FirstName=ROMAN. 5 fields left."

...and so on
```

GPT still makes all decisions. We just clean the desk between decisions.

The memory difference:

```
WITHOUT compaction (current):

Turn 46 receives ALL 45 previous turns as input:

    system_prompt          1,400 tok  (keep)
    tool schemas          5,200 tok  (keep)
    task                   30 tok    (keep)
    turn 1: navigate result 500 tok   \
    turn 2: old snapshot (FULL PAGE TREE) 2,000 tok |
    turn 3: failed upload attempt 1,500 tok | ALL of this
is
    turn 4: another failed attempt 1,200 tok | old junk
that
    ...                    | stays
forever
    turn 25: country dropdown (190 countries) 20,000 tok |
    ...                                     /
    turn 45: latest snapshot 2,000 tok  (only useful
part)

-----
TOTAL: 53,654 tokens
```

WITH compaction (Lever 4):

Turn 46 receives a compact summary + latest state:

system_prompt	1,400 tok	
tool schemas	5,200 tok	
task	30 tok	
rolling summary: "Photo done. 6/7 fields	200 tok	<-- replaces 44
turns		
filled. 2 bugs found. Branch=COUVA."		
turn 45: latest snapshot	2,000 tok	

	TOTAL: ~9,000 tokens	

The rolling summary grows slowly, raw history grows fast:

Summary (Lever 4):

Turn 1: "Navigated to URL"	~10
tokens	
Turn 5: "Navigated. Photo uploaded."	~15
tokens	
Turn 12: "Photo done. FirstName=ROMAN. Middle=A."	~30
tokens	
Turn 16: "Photo done. All 6 fields filled. Branch=COUVA."	~40
tokens	
Turn 46: "All 7 fields done. 2 bugs found. XPath's extracted."	~50
tokens	

Grows by ~1-5 tokens per turn

Raw history (current):

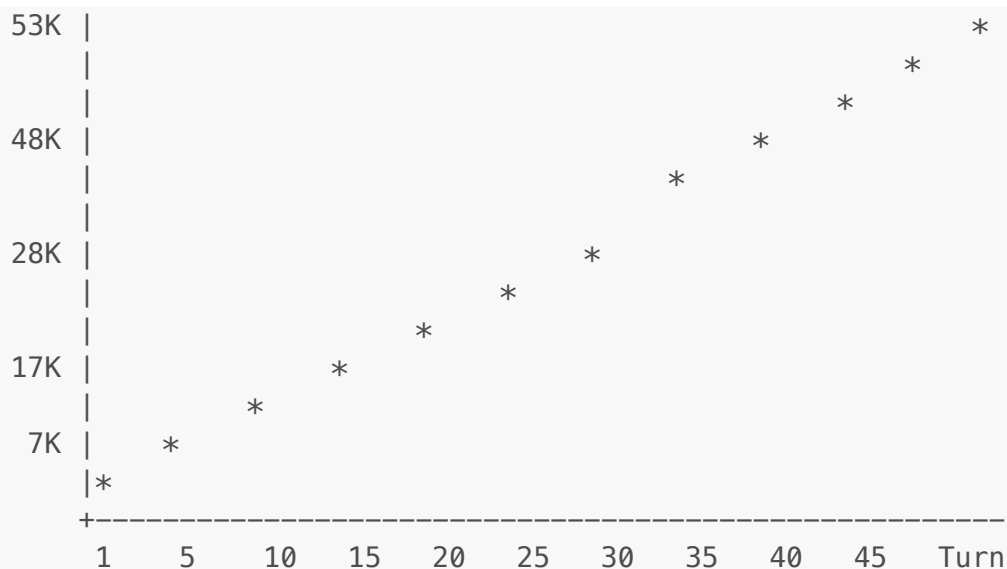
Turn 1:	500 tokens
Turn 5:	5,000 tokens
Turn 12:	16,000 tokens
Turn 16:	17,500 tokens
Turn 46:	53,654 tokens

Grows by ~500-20,000 tokens per turn

Context size over time:

WITHOUT compaction (current):

Tokens



WITH compaction (Lever 4):



Flat line instead of a staircase. Same agent, same results, fraction of the cost.

Code sketch:

```
input_items = [task]
for turn in range(max_turns):
    result = await Runner.run(agent, input=input_items, max_turns=1)

    # COMPACT: replace old history with rolling summary
    input_items = compact_history(result.to_input_list())

    # INJECT: state tracker update ("5/7 fields done, 1 failure")
    input_items = inject_state(input_items, tracker.get_summary())

    # BUDGET: stop if cost approaching limit
    if cost_so_far > budget * 0.8:
        break
```

This unlocks:

- History compaction (remove old snapshots, keep only latest)
- StateTracker integration (already built, just needs wiring)

- Real-time budget enforcement
- Foundation for model tiering (Lever 5)

Lever 5: Model Tiering

Effort: High | **Savings:** 50-70% on simple actions | **Requires Lever 4**

Action Type	Model	Input \$/MTok	Output \$/MTok
Simple fill/click	gpt-5-mini	\$0.25	\$2.00
Dropdowns/uploads/diagnosis	gpt-5	\$1.25	\$10.00

Most turns are simple fills — 80% cheaper with mini. Complex actions stay on GPT-5.

Projected Costs (Full 6-Page Run, 153 Steps)

Version	Approach	Est. Turns	Est. Cost	Est. Time
v1 (current)	SDK black box, verbose prompt	250-300	\$3-5	~25 min
v2	Prompt opt + manual loop + compaction	180-220	\$1.5-2.5	~20 min
v3	+ model tiering	180-220	\$0.8-1.5	~20 min

Next Steps

1. **Apply Levers 1-3** — Prompt-only changes. Zero code risk. Run and compare.
2. **Build Lever 4** — Manual turn loop. Wire in StateTracker. Test on Page 1.
3. **Full run** — Test all 6 pages with v2 architecture.
4. **Lever 5** — Model tiering if further cost reduction needed.